Revisión 1.1

Universidad Nacional de Río Negro

Laboratorio de Informática aplicada

Apunte curso “Introducción a Docker”

Unidad 1

Docentes:

- Lic. Horacio Munoz Abbate
  - (hmunoz@unrn.edu.ar)

Introducción a Docker **2**
¿Qué es Docker? 2
Comparación de contenedores y máquinas virtuales 3
Arquitectura de Docker **5**
Docker Engine 5
Docker Client 5
Docker Registries 6
Docker Objects 6
Docker Compose 7
Docker Swarm 8
Bibliografía **11**

Docker

Introducción a Docker

Que es Docker[1]?

Docker es una aplicación independiente de código abierto que funciona como un motor utilizado para ejecutar aplicaciones en contenedores. Se instala en su sistema operativo (SO), preferiblemente en Linux, pero también se puede instalar en Windows y macOS, que a su vez se ejecuta en una máquina física o virtual.

Una aplicación que se ejecuta en un contenedor está aislada del resto del sistema y de otros contenedores, pero da la ilusión de ejecutarse en su propia instancia del sistema operativo. Se pueden ejecutar varios contenedores Docker en el único sistema operativo simultáneamente.

Un **contenedor** es una unidad de software estándar que empaqueta el código y todas sus dependencias para que la aplicación se ejecute de manera rápida y confiable de un entorno informático a otro. Una imagen de contenedor Docker es un paquete de software

[1] Paquete de software en unidades estandarizadas para desarrollo, envio e implementación (Una unidad de software estandarizada).

ligero, independiente y ejecutable que incluye todo lo necesario para ejecutar una aplicación: código, herramientas del sistema, librerías del sistema y configuraciones.

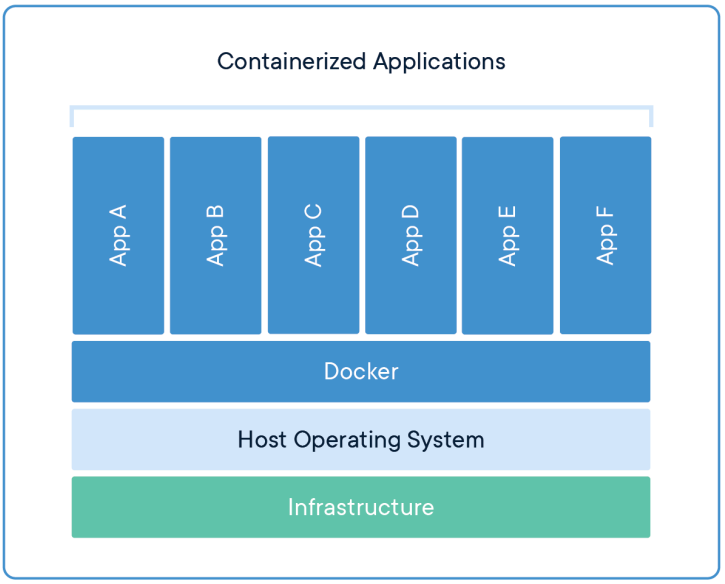
Las **imágenes** se convierten en contenedores en tiempo de ejecución y, en el caso de los contenedores Docker, las imágenes se convierten en contenedores cuando se ejecutan en Docker Engine. Disponible para aplicaciones basadas en Linux y Windows, el software en contenedores siempre se ejecutará igual, independientemente de la infraestructura. Los contenedores aíslan el software de su entorno y aseguran que funcione de manera uniforme a pesar de las diferencias, por ejemplo, entre el desarrollo y producción.

Características de contenedores que se ejecutan en **Docker Engine**:

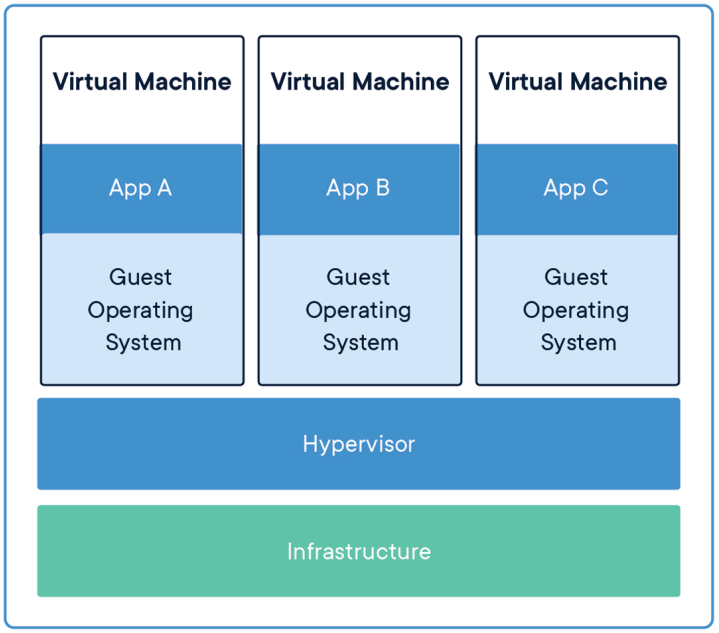
- **Estándar**: Docker creó el estándar de la industria para contenedores, por lo que podrían ser portables en cualquier lugar.
- **Ligero**: los contenedores comparten el núcleo del sistema operativo de la máquina y, por lo tanto, no requieren un sistema operativo por aplicación, lo que aumenta la eficiencia del servidor y reduce los costos de servidor y licencias
- **Seguro**: las aplicaciones son más seguras en contenedores y Docker proporciona las capacidades de aislamiento predeterminadas más sólidas de la industria

Comparación de contenedores y máquinas virtuales

Los contenedores y las máquinas virtuales tienen beneficios similares de aislamiento y asignación de recursos, pero funcionan de manera diferente porque los contenedores virtualizan el sistema operativo en lugar del hardware. Los contenedores son más portátiles y eficientes.



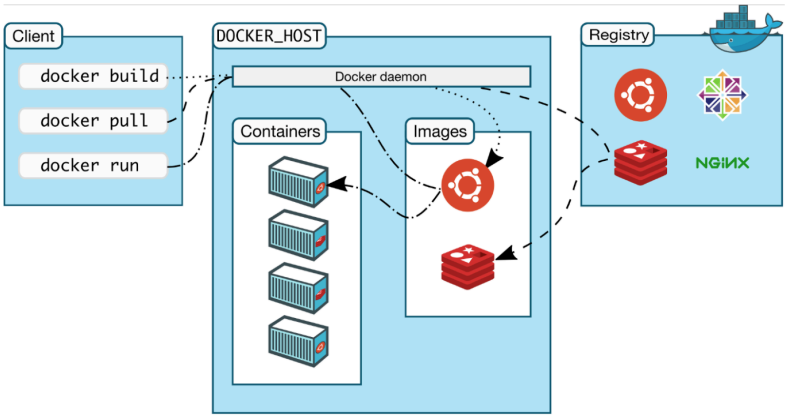
Los contenedores son una abstracción en la capa de la aplicación que agrupa el código y las dependencias juntas. Se pueden ejecutar varios contenedores en la misma máquina y compartir el núcleo del sistema operativo con otros contenedores, cada uno de los cuales se ejecuta como procesos aislados en el espacio del usuario. Los contenedores ocupan menos espacio que las máquinas virtuales (las imágenes de los contenedores suelen tener un tamaño de decenas de MB), pueden manejar más aplicaciones y requieren menos máquinas virtuales y sistemas operativos



Las máquinas virtuales (VM) son una abstracción del hardware físico que convierte un servidor en muchos servidores. El hipervisor permite que varias máquinas virtuales se ejecuten en una sola máquina. Cada VM incluye una copia completa de un sistema operativo, la aplicación, los binarios y bibliotecas necesarias, que ocupan decenas de GB. Las máquinas virtuales también pueden ser lentas para arrancar.

Arquitectura de Docker

La arquitectura de Docker se compone de los siguientes elementos como se puede ver en la figura:



Docker Engine

Es la parte central de todo el sistema Docker. Docker Engine es una aplicación que sigue la arquitectura cliente-servidor. Está instalado en la máquina host. Hay tres componentes en el motor Docker:

- **Servidor**: es el **docker daemon** llamado también **dockerd**. Puede crear y administrar imágenes de docker. Contenedores, redes, etc.
- **Rest API**: se utiliza para indicarle a Docker Daemon qué hacer.
- **Interfaz de línea de comandos** (CLI): es un cliente que se utiliza para ingresar comandos de Docker.

Docker Client

Los usuarios de Docker pueden interactuar con Docker a través de un cliente. Cuando se ejecuta cualquier comando de docker, el cliente los envía al demonio de **dockerd**, que

los ejecuta. Docker API es utilizado por los comandos de Docker. El cliente Docker puede comunicarse con más de un demonio.

Docker Registries

Es la ubicación donde se almacenan las imágenes de Docker. Puede ser un registro de acoplador **público** o un registro de acoplador **privado**. Docker Hub es el lugar predeterminado de las imágenes de Docker, el registro público de sus tiendas. También puede crear y ejecutar su propio registro **privado**.

Cuando ejecuta los comandos docker **pull** o docker **run**, la imagen de docker requerida se extrae del registro configurado. Cuando ejecuta el comando **push** docker, la imagen del docker se almacena en el registro configurado.

Docker Objects

- **Images**: Las imágenes de Docker son plantillas de solo lectura con instrucciones para crear un contenedor de Docker. La imagen de Docker se puede extraer de un concentrador de Docker y usarse tal cual, o puede agregar instrucciones adicionales a la imagen base y crear una imagen de docker nueva y modificada. Puede crear sus propias imágenes utilizando un dockerfile. Cree un dockerfile con todas las instrucciones para crear un contenedor y ejecutarlo; creará su imagen de acoplador personalizada. La imagen de Docker tiene una capa base que es de solo lectura, y la capa superior se puede escribir. Cuando edita un dockerfile y lo reconstruye, solo la parte modificada se reconstruye en la capa superior.
- **Containers**: Después de ejecutar run docker imagen, crea un docker container. Todas las aplicaciones y su entorno se ejecutan dentro de este contenedor. Puede usar Docker API o CLI para iniciar, detener y eliminar un contenedor de Docker.
- **Volumes**: Los datos persistentes generados por Docker y utilizados por los contenedores Docker se almacenan en volúmenes. Docker los administra por completo a través de la CLI de docker o la API de Docker. En lugar de conservar los datos en la capa de escritura de un contenedor, siempre es una buena opción usar volúmenes para ello. El contenido del volumen existe fuera del ciclo de vida de un contenedor, por lo que el uso del volumen no aumenta el tamaño de un contenedor.

- **Networks:** La conexión en red de Docker es un paso a través del cual se comunican todos los contenedores aislados. Hay principalmente cinco controladores de red en Docker.
    1. **Bridge**: es el controlador de red predeterminado para un contenedor. Utiliza esta red cuando su aplicación se ejecuta en contenedores independientes, es decir, múltiples contenedores que se comunican con el mismo host de Docker.
    2. **Host**: este controlador elimina el aislamiento de red entre los contenedores docker y el host docker. Se utiliza cuando no necesita ningún aislamiento de red entre el host y el contenedor.
    3. **Overlay**: esta red permite que los servicios de enjambre se comuniquen entre sí. Se utiliza cuando los contenedores se ejecutan en diferentes hosts Docker o cuando los servicios de enjambre están formados por múltiples aplicaciones.
    4. **Ninguno**: este controlador desactiva todas las redes.
    5. **Macvlan**: Algunas aplicaciones, especialmente aplicaciones heredadas o aplicaciones que monitorean el tráfico de red, esperan estar directamente conectadas a la red física. En este tipo de situación, puede usar el controlador de red macvlan para asignar una dirección MAC a la interfaz de red virtual de cada contenedor, haciendo que parezca una interfaz de red física directamente conectada a la red física. En este caso, debe designar una interfaz física en su host Docker para usar en el macvlan, así como la subred y la puerta de enlace del macvlan. Incluso puede aislar sus redes macvlan utilizando diferentes interfaces de red físicas.

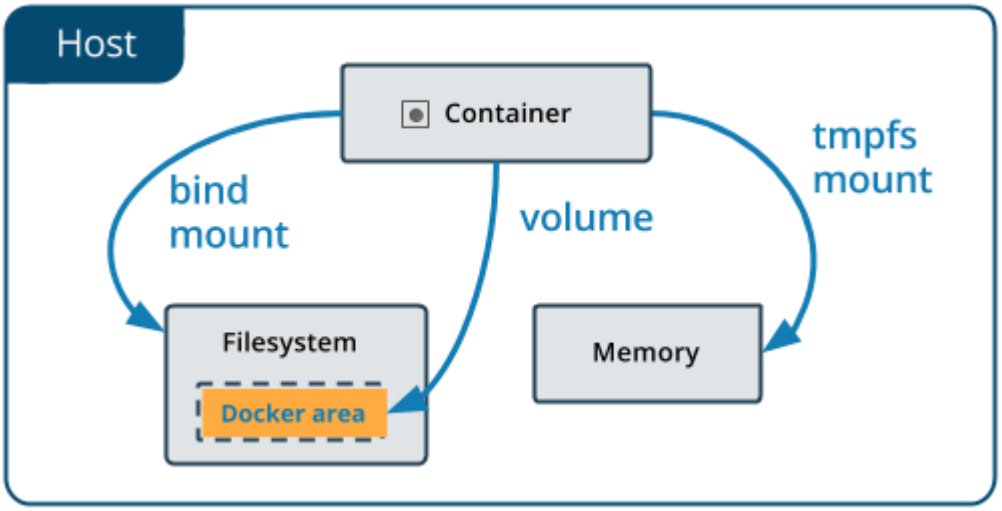
Volúmenes y puntos de Montaje

Los **volúmenes** son el mecanismo preferido para los datos persistentes generados y utilizados por los contenedores Docker. Si bien los montajes de enlace dependen de la estructura de directorios de la máquina host, Docker administra completamente los volúmenes. Los volúmenes tienen varias ventajas sobre los montajes de unión:

1. Los volúmenes son más fáciles de respaldar o migrar que los montajes de unión.

2. Puede administrar volúmenes utilizando los comandos de Docker CLI o la API de Docker.
3. Los volúmenes funcionan tanto en contenedores de Linux como de Windows.
4. Los volúmenes se pueden compartir de manera más segura entre múltiples contenedores.
5. Los controladores de volumen le permiten almacenar volúmenes en hosts remotos o proveedores en la nube, para cifrar el contenido de los volúmenes o para agregar otra funcionalidad.
6. Los volúmenes nuevos pueden tener su contenido rellenado previamente por un contenedor.
7. Además, los volúmenes son a menudo una mejor opción que los datos persistentes en la capa de escritura de un contenedor, porque un volumen no aumenta el tamaño de los contenedores que lo usan y el contenido del volumen existe fuera del ciclo de vida de un contenedor determinado.

Los **montajes de enlace** tienen una funcionalidad limitada en comparación con los volúmenes. Cuando utiliza un montaje de enlace, un archivo o directorio en la máquina host se monta en un contenedor. Se hace referencia al archivo o directorio por su ruta completa o relativa en la máquina host. Por el contrario, cuando usa un volumen, se crea un nuevo directorio dentro del directorio de almacenamiento de Docker en la máquina host, y Docker administra el contenido de ese directorio.



Ejemplos:

- --mount source=nombreVolumen,target=/app (Volumen)
- --mount type=bind,source="$(pwd)"/target,target=/app (bind)

https://docs.docker.com/storage/bind-mounts/
https://docs.docker.com/storage/volumes/

Docker Compose

Es una herramienta que permite la orquestación de contenedores, se utiliza para ejecutar aplicaciones Docker de contenedores múltiples. Por medio de un archivo de configuración, llamando **docker compose** con formato YAML, permite definir aplicaciones de contenedores múltiples en lugar de configurar archivos Docker separados manualmente para cada contenedor. Después de configurar el único archivo YAML, puede ejecutar todos los contenedores necesarios con un solo comando en su consola. Utilizando Docker Compose comenzamos los primeros pasos para la orquestación de contenedores.

Todos los contenedores en los archivos YAML se pueden administrar con comandos simples como los siguientes:

- **$: docker-compose up -d**: inicia todos los contenedores. -d significa que el los contenedores se ejecutan en segundo plano, sin bloquear el Terminal desde donde se ejecutó el comando.
- **$: docker-compose down**: detiene y elimina todos los contenedores.
- **$: docker-compose -f --tail = 0**: imprime mensajes de registro de Todos los contenedores. -f significa que el comando no completa, y en su lugar espera nuevos mensajes de registro. - tail = 0 significa que no queremos ver ningún registro anterior mensajes, solo nuevos.

Docker Swarm

Cuando tenemos un solo Docker Host, es posible administrar los contenedores fácilmente con docker compose, como lo vimos en el apartado anterior. Pero cuando necesitamos escalar en una infraestructura más grande con un número mayor de Docker Host, administrar los contenedores manualmente puede ser difícil, y generalmente es mejor

seleccionar una herramienta de administración centralizada o una solución de orquestación como **Docker Swarm** o **Kubernetes**.

Docker Swarm, es una herramienta de agrupación nativa para Docker que puede convertir un grupo de hosts Docker en un único host virtual. Está totalmente integrado con Docker Engine y le permite utilizar API estándar y procesos de red; está destinado a desplegar, administrar y escalar contenedores Docker. Por lo tanto Swarm es un grupo de hosts Docker que se llaman nodos o cluster.

A continuación definimos los componentes principales de un clúster cuando utilice Docker Swarm:

- Despliegue de clúster

Tiene una API estándar de Docker le permite implementar un clúster con Swarm mediante el uso de una CLI de Docker que facilita la implementación. La facilidad de implementación de Swarm en comparación a otros soluciones como por ejemplo Kubernetes, se logra mediante la capacidad de un solo nodo Maestro (Manager) para decidir cómo distribuir los servicios en el cluster.

- Escalabilidad

Gracias a la API de Docker, la implementación de contenedores y la actualización de servicios tanto en grupos grandes como pequeños se puede hacer de forma rapada. Una interfaz de línea de comando (CLI) es simple y fácil de entender.

- Balanceo de carga

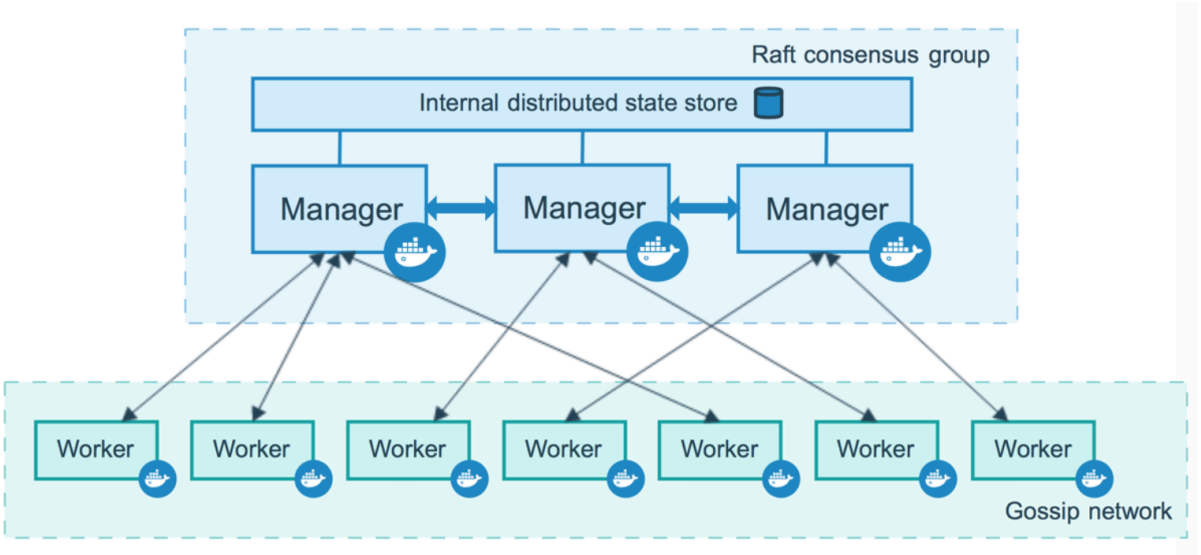
El equilibrio de carga es una función incorporada y se puede realizar automáticamente utilizando la red interna Swarm. Todas las solicitudes al clúster se distribuyen y se redirigen a los nodos del clúster; cualquier nodo puede conectarse a cualquier contenedor. Docker Swarm utiliza un elemento DNS para distribuir las solicitudes entrantes a los nombres de servicio.

- Crear y ejecutar contenedores

La gran mayoría de los comandos disponibles para Docker CLI se pueden utilizar cuando se utiliza Docker Swarm, gracias a la API estandarizada de Docker Swarm. Docker Compose define el trabajo con volúmenes y redes usadas, además de definir qué contenedores deben agruparse. El número preciso de réplicas se puede configurar con Docker Compose para Docker Swarm.

- Redes

Hay una red interna predeterminada para la comunicación de contenedores dentro de un clúster implementado con Swarm, en la que se pueden agregar más redes si es necesario.



En la figura anterior se puede ver cómo interactúan los nodos Manager y Worker para gestionar y administrar los servicios que están desplegados en el cluster. Cada Docker Host que pertenece al cluster debe tener uno de estos dos roles Manager o Worker, también es posible promover un nodo Worker a Manager.

En caso que un Worker tenga problemas, todos los contenedores que estaban dentro de este Host, se van a balancear sobre el resto de Workers presentes en el Cluster. (Animación al eliminar un Worker)

Bibliografía

- https://docs.docker.com/get-started/overview/
- https://www.docker.com/resources/what-container
- https://geekflare.com/docker-architecture/
- Learning Docker ISBN 978-1-78439-793-7
    - https://drive.google.com/file/d/1MeIM5LeRF6GuHVe15pyn7irr0UQzyCLM/view?usp=sharing
- Hands-On Microservices with Spring Boot and Spring Cloud
    - ISBN 978-1-78961-347-6
    - https://drive.google.com/file/d/1Qx7hMyxdVOtQUuYC48VYErss5Z5iNv-1/view?usp=sharing
- https://spring.io/guides/topicals/spring-boot-docker/